Projeto: Partida Sequencial de 4 Motores — Gabarito

> **Uso:** documento de referência para o professor conferir/corrigir o exercício. A versão que vai para o aluno é partida-sequencial-motores-enunciado.md, sem o código-solução.
>
> **Status:** projeto de papel — ainda sem montagem física nem capturas de tela.

1. Objetivo

Ligar 4 motores em sequência a partir de um único botão Start, cada um com um atraso programado em relação ao anterior, e permitir que um botão Parar corte tudo imediatamente a qualquer momento — reiniciando a sequência do zero num próximo Start.

---

2. Resposta à pergunta central: Function ou Function Block?

Function Block. Três motivos, todos ligados direto à aula 09:

- A lógica **precisa guardar estado** entre ciclos de varredura: em que ponto da sequência está, e há quanto tempo cada etapa começou. Uma Function, por definição, não guarda nada — cada chamada é isolada.
- A lógica **produz várias saídas ao mesmo tempo** (`motor1`, `motor2`, `motor3`, `motor4`, e opcionalmente um `ativo` indicando sequência em andamento). Uma Function tem no máximo uma saída de dado.
- Ela **usa por dentro outros Function Blocks com memória própria** (`TON`, `SR`, `R_TRIG`) — exatamente o padrão de composição já visto em `meu_contador` (aula 09), que usava um `ADD` e uma Function (`meu_move`) por dentro. Aqui a composição é de Function Blocks dentro de um Function Block maior.

Batizamos o bloco de `PARTIDA_SEQUENCIAL`.

---

3. A sacada de projeto: temporizadores em paralelo, não em cadeia

A forma mais simples de implementar isso **não** é um contador de passos (`passo := passo + 1`) comandando um `CASE`. É mais simples (e mais fácil de zerar corretamente

no Parar) usar **três temporizadores `TON` independentes, todos disparados ao mesmo instante**, cada um com um tempo-alvo diferente **medido a partir do Start**:

Temporizador	`PT` (tempo desde o Start)	Liga
`t2`	`pt_m2` (ex.: `T#2s`)	`motor2`
`t3`	`pt_m3` (ex.: `T#4s`)	`motor3`
`t4`	`pt_m4` (ex.: `T#6s`)	`motor4`

`motor1` liga direto, sem temporizador, assim que a sequência é armada.

Os três `TON` recebem o **mesmo** sinal de habilitação (`ativo`). Quando `ativo` cai (por causa do Parar), os três `IN` caem juntos — e por definição de `TON`, isso zera `Q` **e** `ET` imediatamente, então um novo Start já começa com os temporizadores limpos, sem precisar zerar nada "na mão".

O "armar/desarmar" de `ativo` é, ele mesmo, um latch Set/Reset — o mesmo `SR` da aula 06/09 — combinado com um `R_TRIG` para transformar o nível do botão Start num pulso (senão o `SR` ficaria sendo re-setado a cada ciclo enquanto o botão ficasse pressionado, o que não é problema aqui já que `S1` repetido não muda nada, mas o padrão correto é sempre tratar botão como pulso).

---

4. Hardware e pinagem

Papel	Componente	Pino	Endereço
Start da sequência	Botão pulsador (NA + pull-down 10kΩ)	GPIO23	`%IX0.0`
Parar tudo	Botão pulsador (NA + pull-down 10kΩ)	GPIO22	`%IX0.1`
Motor 1	LED + resistor (ou relé + motor)	GPIO2	`%QX0.0`
Motor 2	LED + resistor (ou relé + motor)	GPIO4	`%QX0.1`
Motor 3	LED + resistor (ou relé + motor)	GPIO16	`%QX0.2`
Motor 4	LED + resistor (ou relé + motor)	GPIO17	`%QX0.3`

---

5. O Function Block `PARTIDA_SEQUENCIAL`

```
FUNCTION_BLOCK PARTIDA_SEQUENCIAL
VAR_INPUT
    start : BOOL;   (* nivel do botao — a deteccao de borda e' feita aqui dentro *)
    parar : BOOL;   (* nivel do botao — desliga tudo enquanto estiver ligado *)
    pt_m2 : TIME;   (* tempo, contado a partir do start, ate o motor 2 ligar *)
    pt_m3 : TIME;   (* tempo, contado a partir do start, ate o motor 3 ligar *)
    pt_m4 : TIME;   (* tempo, contado a partir do start, ate o motor 4 ligar *)
END_VAR
VAR_OUTPUT
    motor1 : BOOL;
    motor2 : BOOL;
    motor3 : BOOL;
    motor4 : BOOL;
    ativo  : BOOL;  (* sequencia armada/em andamento *)
END_VAR
VAR
    pulso_start : R_TRIG;
    seq_armada  : SR;
    t2          : TON;
    t3          : TON;
    t4          : TON;
END_VAR

(* start (nivel) -> pulso -> latch, com reset dominante pelo parar *)
pulso_start(CLK := start);
seq_armada(S1 := pulso_start.Q, R := parar);
ativo := seq_armada.Q1;

(* motor 1 liga junto com o armar da sequencia *)
motor1 := ativo;

(* motores 2, 3 e 4: temporizadores independentes, todos a partir de "ativo" *)
t2(IN := ativo, PT := pt_m2);
motor2 := t2.Q;

t3(IN := ativo, PT := pt_m3);
motor3 := t3.Q;

t4(IN := ativo, PT := pt_m4);
motor4 := t4.Q;

END_FUNCTION_BLOCK
```

- Quando `parar` vai para `1`: `seq_armada` reseta (`SR` = Reset disponível como entrada dedicada, sempre vence enquanto `parar` estiver em `1`), `ativo` cai, `motor1` cai, e os três `TON` têm `IN` caindo junto — `Q` e `ET` dos três zeram no mesmo ciclo.

- Um novo `start` só volta a armar a sequência depois que `parar` for solto (comportamento natural de um `SR` / `RS`: enquanto `R` estiver em `1`, nenhum `S1` consegue setar).

---

6. Programa principal (`main`)

```
PROGRAM main
VAR
    (* entradas fisicas *)
    start_botao : BOOL AT %IX0.0;
    parar_botao : BOOL AT %IX0.1;

    (* saidas fisicas *)
    motor1 : BOOL AT %QX0.0;
    motor2 : BOOL AT %QX0.1;
    motor3 : BOOL AT %QX0.2;
    motor4 : BOOL AT %QX0.3;

    (* instancia do bloco *)
    SEQ0 : PARTIDA_SEQUENCIAL;
END_VAR

SEQ0(
    start := start_botao,
    parar := parar_botao,
    pt_m2 := T#2s,
    pt_m3 := T#4s,
    pt_m4 := T#6s
);

motor1 := SEQ0.motor1;
motor2 := SEQ0.motor2;
motor3 := SEQ0.motor3;
motor4 := SEQ0.motor4;

END_PROGRAM
```

- Os três tempos (`T#2s`, `T#4s`, `T#6s`) ficam **num único lugar**, na chamada da instância — exatamente o pedido do item 5 do enunciado. Trocar o espaçamento entre partidas é mudar três literais, sem tocar no corpo do Function Block.
- Espaçamento uniforme de 2 segundos entre motores nesse exemplo, mas os três parâmetros são independentes — nada obriga que sejam igualmente espaçados.

---

7. Bônus: duas linhas de motores independentes

Respondendo à pergunta-guia 6 do enunciado: como `PARTIDA_SEQUENCIAL` é um Function Block, nada impede declarar uma segunda instância — `SEQ1 : PARTIDA_SEQUENCIAL;` — ligada a outro botão Start e outros 4 GPIOs, controlando um segundo grupo de motores **de forma totalmente independente** do primeiro. É o mesmo argumento de `MEU_CONTADOR0` e `MEU_CONTADOR1` na aula 09: cada instância guarda a sua própria memória (seu próprio `seq_armada`, seus próprios `t2` / `t3` / `t4`). Se `PARTIDA_SEQUENCIAL` fosse uma Function, essa reutilização com estados independentes seria impossível.

---

8. Roteiro de testes em bancada

1. Pressione Start — `motor1` liga na hora.
2. Aguarde 2s — `motor2` liga.
3. Aguarde até completar 4s (desde o Start) — `motor3` liga.
4. Aguarde até completar 6s (desde o Start) — `motor4` liga; a partir daí os 4 motores ficam ligados.
5. Com a sequência completa, pressione Parar — os 4 motores desligam no mesmo ciclo.
6. Pressione Start de novo — a sequência **inteira** se repete desde o motor 1 (confirmar que nenhum temporizador "lembrou" tempo da rodada anterior).
7. Pressione Start, espere só 1s (antes do motor 2 ligar) e pressione Parar — tudo desliga; confirme em modo debug que `t2.ET`, `t3.ET` e `t4.ET` voltam a `0`.
8. Com a sequência rodando, pressione Start de novo no meio (por exemplo, com só motor1 e motor2 ligados) — nada deve mudar no andamento; a sequência continua normalmente até completar.

---

9. Próximos passos

- [ ] Montar o circuito (ou confirmar os 4 LEDs simulando os motores).
- [ ] Capturar telas do projeto no OpenPLC Editor (variáveis do FB, corpo do `PARTIDA_SEQUENCIAL`, instância no `main`, Pin Mapping).
- [ ] Rodar o roteiro de testes da seção 8 e registrar os resultados (com prints do modo debug mostrando `t2.ET` / `t3.ET` / `t4.ET`).
- [ ] Opcional: implementar o bônus da seção 7 (duas instâncias independentes) e documentar.
- [ ] Criar o `quadro-resumo.md` depois que a lógica estiver validada na bancada.